

EXP.NO: 9

FirstFit&Best Fit Memory Allocation

DATE: 20/03/2026

AIM:

To implement and compare contiguous memory allocation strategies — First Fit and Best Fit — to allocate memory blocks to processes and analyze memory utilization.

ALGORITHM:

1. Start the experiment.
2. Input the sizes of memory blocks and process sizes.
3. First Fit: For each process, scan memory blocks from the beginning and allocate the first block large enough to satisfy the request. Reduce the block size accordingly.
4. Best Fit: For each process, scan all blocks and find the smallest block that is large enough. Allocate that block and reduce its size, minimizing wasted space.
5. For each algorithm, display which block is allocated to each process.
6. Display any processes that could not be allocated and remaining free space in each block.
7. Stop the experiment.

CODE:

```
#include <stdio.h>

#define MAX 10

/* ■■ First Fit ■■ */
void firstFit(int bsize[], int m, int psize[], int n) {
    int alloc[MAX];
    int blocks[MAX];
    for (int i = 0; i < m; i++) blocks[i] = bsize[i];
    for (int i = 0; i < n; i++) alloc[i] = -1;

    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++)
            if(blocks[j]>=psize[i]) {
                alloc[i] = j;
                blocks[j] -= psize[i];
                break;
            }

    printf("\n-- First Fit --\n");
    printf("Process   Size   Allocated Block\n");
    for (int i = 0; i < n; i++) {
        if (alloc[i] != -1)
            printf("  P%d        %dKB      Block %d (rem: %d KB)\n",
                    i+1, psize[i], alloc[i]+1, blocks[alloc[i]]);
        else
            printf("  P%d        %dKB      Not Allocated\n", i+1, psize[i]);
    }
}
```

```

}

/* ■■ Best Fit ■■ */
void bestFit(int bsize[], int m, int psize[], int n) {
    int alloc[MAX];
    int blocks[MAX];
    for (int i = 0; i < m; i++) blocks[i] = bsize[i];
    for (int i = 0; i < n; i++) alloc[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++)
            if (blocks[j] >= psize[i]
                if(bestIdx==-1|| blocks[j] < blocks[bestIdx])
                    bestIdx = j;
        if (bestIdx != -1) {
            alloc[i] = bestIdx;
            blocks[bestIdx] -= psize[i];
        }
    }

    printf("\n-- Best Fit --\n");
    printf("Process  Size  Allocated Block\n");
    for (int i = 0; i < n; i++) {
        if (alloc[i] != -1)
            printf("  P%d      %dKB      Block %d (rem: %d KB)\n",
                    i+1, psize[i], alloc[i]+1, blocks[alloc[i]]);
        else
            printf("  P%d      %dKB      Not Allocated\n", i+1, psize[i]);
    }
}

int main() {
    int blocks[] = {100, 500, 200, 300, 600};
    int procs[] = {212, 417, 112, 426};
    int m = 5, n = 4;

    firstFit(blocks, m, procs, n);
    bestFit (blocks, m, procs, n);
    return 0;
}

```

OUTPUT:

```

[hp@localhost ~]$ gcc memory.c -o memory
[hp@localhost ~]$ ./memory

-- First Fit --
Process Size Allocated Block
  P1      212 KB   Block 2 (rem: 288 KB)
  P2      417 KB   Block 5 (rem: 183 KB)
  P3      112 KB   Block 2 (rem: 176 KB)
  P4      426 KB   Not Allocated

-- Best Fit --

```

Process	Size	Allocated	Block
P1	212 KB	Block 4	(rem: 88 KB)
P2	417 KB	Block 5	(rem: 183 KB)
P3	112 KB	Block 3	(rem: 88 KB)
P4	426 KB	Not Allocated	

RESULT:

Thus, First Fit and Best Fit memory allocation strategies were successfully implemented. Best Fit minimizes wasted space per allocation while First Fit offers faster allocation by selecting the first available suitable block.